

Audit of MemGraphRAG and MG²-RAG: can we build a visual MemGraphRAG by 23 October?

2AMM20 Research Topics in Data Mining — Understanding LLMs track · feasibility audit

28 September 2026

Evidence legend. Every claim carries a tag.

- **[V-run]**: verified by executing code, either the repos or stubbed copies of them. Logs are in `audit/logs/`.
- **[V-read]**: verified by reading the code or paper at the cited location.
- **[I]**: inferred. This covers reasoning, external facts I did not check, or estimates.

Paths are relative to each repository root: `MemGraphRAG/...` or `MG2-RAG/...`. Paper references are *MemGraphRAG paper* = arXiv 2606.00610v1 and *MG² paper* = arXiv 2604.04969v2. Neither repository was modified. All execution happened on copies in `audit/work/` [V-run].

1. Summary and recommendation

Bottom line: yes, the combination is feasible in three weeks, but only in its “late-fusion” form. That means adding MG²-RAG’s image and grounding layer onto MemGraphRAG’s final memory graph (Option A, §5), rather than rebuilding MemGraphRAG’s memory and agents to be multimodal. The main risks are not in the code. They are data (images), GPU environment and budget, and the fact that MemGraphRAG’s published code implements less than its paper.

What the audit found:

1. **Both repos derive from HippoRAG and share the same graph plumbing.** Both use MD5 node IDs with `chunk- /entity-` prefixes, parquet embedding stores, igraph, and PPR ranking over passage/chunk nodes (§4) [V-read]. This makes graph-level integration cheap. Both core pipelines ran end-to-end on my stubbed copies (§3) [V-run].
2. **MemGraphRAG’s code implements the indexing half of its paper, but its retrieval is essentially HippoRAG2.**
 - Implemented: schema extraction, frequency filtering, and LLM-based conflict detection and resolution. These are sequential batch LLM passes, not agents.
 - Not implemented [V-run, V-read]:
 - The paper’s schema retrieval.
 - Type-node initialisation with hub suppression (Eq. 7).
 - The IDF information-density term (Eq. 8).
 - The containment and LLM-judge accuracy metrics.
 - Type nodes receive **zero** PPR seed mass [V-run].
 - The bundled launch scripts fail immediately [V-run], and the QA runner computes no metrics at all [V-read].
3. **MG²-RAG implements its method closely.** It is, however, a GPU-only research artefact:
 - It needs two GPUs by default, CuPy, spaCy on GPU, a 33 GB EVA-CLIP-8B checkpoint, and a **manually gated** SAM3 checkpoint.
 - It ships no dataset loaders or metrics; the only runnable example is a 5-document demo.
 - Its visual grounding only prompts SAM3 with *named entities* of six spaCy types. For entity-centric photos (animals, plants), the main object is often not such an entity. The authors’ own showcase only grounds “impala” because spaCy mislabelled it as PERSON [V-read, V-run with stubs].
4. **The E-VQA and InfoSeek pipelines depend on very large image sources.** The E-VQA builder needs a scan of AToMiC-Images (**174 GB**, gated), and InfoSeek query images come from OVEN (**294 GB**, gated) [V-run: HF metadata API]. A small self-built subset is the realistic path (§6).

Recommendation.

- **Build Option A: “MemGraphRAG-V”.**
 - Add image and region evidence to MemGraphRAG’s graph (passage–image and image–entity edges).

- Choose SAM3 prompts from MemGraphRAG’s LLM-extracted entities, filtered by their *schema types*. This “ontology-guided grounding” is a defensible novelty that fixes MG²-RAG’s named-entity limitation.
- Add a visual seed channel to PPR.
- Estimated effort: 8–12 person-days of coding. Risk: medium.
- **Main baseline:** MemGraphRAG with images converted to captions (Option D, 1–2 days). This is exactly the “translation-to-text” setting both papers argue against.
- **Other baselines:** MG²-RAG as-is, dense CLIP retrieval, and zero-shot MLLM. All exist in MG²-RAG.
- **Evaluation:**
 - One small multimodal KB-VQA subset: about 500 queries over at most 2–5k documents, InfoSeek or E-VQA style, built from M2KR text plus only the needed images.
 - A text-only regression on MemGraphRAG’s bundled HotpotQA/2Wiki/MuSiQue data.
- **Develop with a small CLIP-class encoder** (e.g. SigLIP2-so400m, 4.5 GB) and keep EVA-CLIP-8B for one final run if the budget allows.
- **Decide early how to cite MemGraphRAG.** You will be extending *the code*, which lacks Eqs. 7–8. Either implement them (about 1 day, §5) or state clearly that you build on the released implementation.

Nothing large has been downloaded. The download and cost plan that needs your go-ahead is in §3.3.

2. Paper vs. code

2.1 MemGraphRAG

Paper component	What the code does	Status	Evidence
Multi-agent system (Extraction, Detection and Resolution agents with shared memory, §4.1, App. D.2)	A fixed sequence of batch LLM passes inside <code>index_with_memory</code> . No agent abstraction, asynchronous triggers or iterative loop; “agent” never appears in the code.	Different in form	<code>src/MemGraphRAG.py:965-1034</code> [V-read, V-run]
Composite extraction: schemas, facts and passages jointly per chunk (Eq. 3/13)	HippoRAG OpenIE (one NER call and one triple call per chunk), then a separate ontology-extraction LLM call per fact (up to 3 passages as context).	Different; costly	<code>src/information_extraction/openie_openai.py:173-176</code> ; <code>src/MemGraphRAG.py:369-425</code> ; stub: 17 facts → 17 schema calls [V-run]
Schema promotion when $\text{Freq} \geq \tau$ (Eq. 4/14)	Default mode <code>percentile</code> deletes the lowest 20 % of schemas regardless of absolute frequency. <code>absolute</code> mode (min freq 2) exists.	Different default	<code>src/MemGraphRAG.py:452-485</code> , <code>code/index.py:58-60</code> , <code>utils/config_utils.py:183-185</code> . Stub: two <i>correct</i> facts (Einstein→relativity, Ulm→Danube) dropped [V-run]
Conflict candidates: $\text{Sim} > \delta$ or symbolic match (Eq. 5/15)	Symbolic only: same (subject, relation) with a different object, plus an optional hard-coded reverse-relation list. No embedding similarity. “Streaming” is emulated by comparing each fact only with earlier facts.	Partial	<code>src/MemGraphRAG.py:496-527</code> , 589-590 [V-read]

Paper component	What the code does	Status	Evidence
Evidence-based resolution (Eq. 16, three conflict types)	Implemented per connected component of the conflict graph, with up to 2 source passages per fact (1,200 chars each). Runs sequentially . Modified triples are not re-normalised , so the stub run created a duplicate node (Isaac Newton vs isaac newton).	Implemented, with a bug	:487-494, 676-799, 756; stub “modified_case”: 16 vs 14 entity nodes [V-run]
Graph views (type, entity and passage nodes); type bridging; similarity bridging (§4.2.3)	Implemented in <code>build_memory_graph</code> . Facts are edges, not nodes. Type-to-type edges keep only frequency (relation names dropped). Entity-to-type weight = $1/ \text{entities of type} $.	Implemented	:801-923 [V-read, V-run]
Stage I: retrieve schemas, facts and passages (§4.3.1, Alg. 2 I.2)	Only facts and passages are scored. The schema layer is never embedded (<code>embedding</code> stays <code>None</code>) or retrieved.	Missing	<code>src/Memory.py:42</code> , <code>src/MemGraphRAG.py:435</code> , 1976-2047 [V-read]
Entity seed = mean fact similarity (Eq. 6)	HippoRAG2 rule: the <i>last</i> matching fact’s min-max-normalised score, divided by the number of passages containing the entity.	Different	:2124-2145 [V-read]
Type seed with hub suppression (Eq. 7)	Type-node keys are collected but never used, so type nodes get zero reset probability.	Missing	:1891-1892; stub PPR seed mass: { <code>passage</code> , <code>entity</code> } only [V-run]
Passage seed with IDF information density (Eq. 8)	$\text{DPR score} \times 0.05$; no IDF term anywhere in the code.	Missing	:2160-2166 [V-read]
PPR, damping 0.5; dense-retrieval fallback when no facts survive	Implemented (<code>igraph prpack</code> , CPU).	Implemented	:2299-2339, 1100-1105 [V-read]
Fact filter threshold; top-k = 5	Raw-cosine threshold with inconsistent defaults: 0.6 (CLI), 0.4 (README), 0.2 (function), 0.5 (config). Optional HippoRAG2 DSPy LLM filter. QA top-k defaults to 10.	Inconsistent	<code>code/retrieval_dataset_test.py:119,241-251,248</code> ; README.md:132; <code>config_utils.py:222-224</code> [V-read]
NV-Embed-v2 embeddings	README uses <code>bge-large-en-v1.5</code> . The BGE wrapper uses mean pooling and drops the query instruction (the BGE model card recommends CLS pooling plus an instruction [I]). NV-Embed-v2 loads without a dtype, i.e. <code>fp32</code> [I].	Different default	README.md:80; <code>src/embedding_model/BGE.py:17,90,112</code> ; <code>NVEmbedV2.py:63</code> [V-read]
Metrics: Str-Acc (containment), LLM-Acc, Context Relevance / Evidence Recall	Only EM, F1 and string-based Recall@k (from HippoRAG). The QA runner calls <code>rag_qa(gold_answers=None)</code> and computes no metric.	Missing	<code>src/evaluation/qa_eval.py</code> , <code>retrieval_eval.py</code> ; <code>code/retrieval_dataset_test.py:127,179</code> [V-read]

Paper component	What the code does	Status	Evidence
Datasets (HotpotQA, 2Wiki, MuSiQue, G-Medical, G-Novel)	Bundled: 1,000 questions each for HotpotQA, 2Wiki and MuSiQue, plus 2,062 G-Medical questions (4 types). G-Novel absent. Corpora are flat <code>.txt</code> files re-chunked into 256-token windows, so gold passage boundaries are lost and Recall@k against gold passages needs re-mapping.	Partial	<code>dataset/*</code> ; <code>code/index.py</code> :16-37,78 [V-run counts, V-read]
Code hygiene	Uses <code>eval()</code> on LLM output and stored strings. 58 committed <code>.pyc</code> files, including <code>HippoRAG.*.pyc</code> . <code>assert False</code> used for control flow. At least nine config fields are never read (e.g. <code>enable_triple_conflict_resolution</code> , <code>graph_type</code>).	Risk	<code>openie_openai.py</code> :33,82; <code>MemGraphRAG.py</code> :212,2225,2252; <code>rerank.py</code> :125; <code>config_utils.py</code> :149-164 [V-read, V-run <code>git ls-files</code>]

2.2 MG²-RAG

Paper component	What the code does	Status	Evidence
spaCy <code>en_core_web_trf</code> NER and dependency-rule relations (§3.1, App. A.1)	Implemented. The rules also emit <code>has_attribute</code> and <code>related_to</code> , which the paper doesn't mention.	Implemented, extended	<code>src/mmgraphrag/information_extraction/ner_spacy.py</code> :59-169, 146, 162 [V-read]
Entity-driven SAM3 grounding, $\sigma > \tau = 0.5$ (Eq. 1)	Implemented, but prompts are limited to entities labelled FAC, LOC, ORG, PERSON, PRODUCT or WORK_OF_ART, not all of E_k . With correct labels, the impala photo is prompted with "africa", "hinrich lichtenstein" and similar, and nothing is grounded. The authors' showcase grounds "impala" only because spaCy tagged it PERSON.	Narrower than paper	<code>MMGraphRAG.py</code> :578-586; <code>examples/showcase/impala/text_structure.json</code> :5; stub run: 1 vs 0 sub-images [V-run]
Modality-preserving fusion (incidence matrices M_{OMI} , M_{SMI})	Objects and sentences are not graph nodes. They live in dictionaries (<code>subimg_to_entity_map</code> , <code>global_sentence_entities_map</code>). An image–entity edge carries the max grounding score.	Implemented (as maps)	<code>MMGraphRAG.py</code> :588-617, 359-431 [V-read, V-run]
Contextual, semantic and grounding edges	Chunk–entity (co-occurrence count), entity–entity (dependency relations, both directions), chunk–image, image–entity. Duplicate edges are summed by <code>simplify</code> .	Implemented	:410-424, 602-617, 680 [V-read]
One EVA-CLIP-8B space for sentences, chunks, images and objects	Implemented, but text is truncated to 77 tokens , so a chunk embedding only sees roughly its first paragraph. Entity embeddings are computed and then discarded at retrieval.	Implemented; limitation	<code>utils/config_utils.py</code> :61, <code>.env.example</code> :82, <code>embedding_model/eva_clip.py</code> :98; <code>MMGraphRAG.py</code> :293,1341-1360 [V-read]

Paper component	What the code does	Status	Evidence
Seed aggregation (Eq. 2) and modality fusion (Eq. 3), top-k then normalise	Implemented with deviations: the sentence→entity score is divided by entity document frequency, then averaged, min-max normalised and top-k'd; object→entity scores are summed, not mean-pooled; all similarity lists are min-max normalised per query.	Implemented, deviates	:885-1008, 909-915, 932-943, 1408-1415 [V-read]
PPR (Eq. 4) with GPU acceleration	CuPy sparse power iteration on a row-normalised adjacency; convergence by L1. No CPU fallback.	Implemented, CUDA-only	:1487-1622 [V-read]
Hyperparameters (App. A.3 Table 7)	<code>.env.example</code> reproduces the InfoSeek column only.	Partial	<code>.env.example</code> [V-read]
Tasks: E-VQA, InfoSeek, ScienceQA, CrisisMMD	Only the InfoSeek QA prompts (text-LLM and MLLM) and an E-VQA knowledge-base builder. No loaders for InfoSeek, ScienceQA or CrisisMMD; ScienceQA/CrisisMMD prompts appear only in the paper.	Mostly missing	<code>src/mmgraphrag/prompts/templates/*</code> ; <code>examples/data/build_evqa_100k.py</code> [V-read]
Metrics: R@K, BEM, accuracy	None implemented; the demo prints <code>gold_rank</code> .	Missing	<code>examples/run_fixed_example.py</code> : 98-115 [V-read]
Baselines	Dense text-only and image-only retrieval plus a “visual pivot” retriever exist; zero-shot QA via <code>test_basemodel=True</code> .	Available	<code>MMGraphRAG.py</code> :1062, 1120, 1181, 1625 [V-read]
Minor bugs	CLIP text cleaning only runs if the first item is a list, i.e. effectively never. Worker results are re-processed after the loop (harmless). <code>synonymy_*</code> config is never used.	Minor	<code>eva_clip.py</code> :92; <code>MMGraphRAG.py</code> :536-543; <code>config_utils.py</code> :74-79 [V-read]

3. Getting each codebase running

3.1 MemGraphRAG

Intended setup (README): Python 3.10, `pip install -r requirements.txt`, an OpenAI-compatible LLM endpoint and a local embedding model; then `code/index.py`, then `code/retrieval_dataset_test.py`.

What actually happens:

Step	Result	Evidence
<code>bash code/run_index.sh</code>	Aborts: PYTHON: unbound variable. \$PYTHON is never defined.	<code>code/run_index.sh</code> :32 [V-run]
<code>bash code/run_retrieval_test.sh</code>	Same abort. Also points at a nonexistent <code>input/smallcorpus/...</code> and hard-codes a third-party proxy (<code>https://apis.aaiife.cn/v1</code>).	<code>run_retrieval_test.sh</code> :14,32,47 [V-run]
README paths	<code>datasets/corpus/...</code> does not exist; the folder is <code>dataset/</code> , and corpora are <code>*.txt</code> files.	<code>README.md</code> :91,123 [V-run]

Step	Result	Evidence
<code>pip install -r requirements.txt</code>	Pins <code>vllm==0.6.6.post1</code> , which is Linux-only and pins <code>torch 2.5.1</code> . vLLM is only needed for offline OpenIE (lazy import).	<code>requirements.txt</code> ; <code>src/MemGraphRAG.py:121</code> [V-read] / Windows wheel availability [I]
<code>import src.MemGraphRAG</code>	Fails without <code>gritlm</code> , even when using BGE. <code>embedding_model/__init__.py</code> imports GritLM eagerly; the README calls it optional. <code>gritlm</code> pulls <code>wandb</code> , <code>mteb</code> , <code>datasets</code> and <code>sentence-transformers</code> .	<code>src/embedding_model/__init__.py:4</code> , <code>README.md:70</code> [V-run]
Retrieval on Windows	<code>igraph 0.11.8</code> Windows wheel writes GraphML but cannot read it (<code>GraphML support is disabled</code>). Retrieval re-reads <code>graph.graphml</code> , so it must run on Linux.	<code>src/MemGraphRAG.py:185</code> [V-run]
Stubbed end-to-end run (my script)	Works. OpenIE → schema extraction → ontology filter → conflict detection → resolution → memory graph → PPR retrieval → QA, using a stub LLM and stub embedder on an 8-passage corpus with a planted conflict (Newton born 1643 vs 1645).	<code>audit/scripts/mock_memgraphrag_e2e.py</code> , <code>audit/logs/mock_memgraphrag/report.json</code> [V-run]
Legacy HippoRAG-style <code>index()</code> path	Also works and gives a “without memory” ablation for free.	<code>src/MemGraphRAG.py:214-275</code> [V-run]

Recipe that should work on Snellius [I, based on the above]:

1. Create a Python 3.10/3.11 venv.
2. Install the requirements without `vllm`, plus `gritlm` (or comment out `embedding_model/__init__.py:4`).
3. Set `OPENAI_API_KEY` (or run a vLLM server) and point `--embedding-model` to a local `bge-large-en-v1.5`.
4. Run `python code/index.py --corpus dataset/hotpotqa/hotpotqa.txt ...`, then `python code/retrieval_dataset_test.py ...` directly, not via the `.sh` files.

My stub run used `torch 2.7.0` and `transformers 4.49.0` (MG²-RAG’s pins) without problems. That shows MemGraphRAG’s own code runs on the newer stack, though real model loading was not exercised [V-run].

3.2 MG²-RAG

Intended setup (README): `conda env` (Python 3.10, `torch 2.7.0` + `CUDA 12.6`), checkpoints EVA-CLIP-8B + SAM3 + `en_core_web_trf`, `.env` from `.env.example`, then `python main.py`, which indexes and retrieves the bundled impala demo.

Item	Finding	Evidence
Demo size	5 documents, 1 knowledge-base image, 1 query. A retrieval sanity check, not an evaluation.	<code>examples/data/impala_demo/input.json</code> [V-read]
GPUs	Default layout is 2 GPUs: GPU 0 for EVA-CLIP, dense search and PPR; GPU 1 for spaCy and SAM3. On one GPU, set <code>MG2RAG_SAM3_DEVICE=0</code> (spaCy is forced to the same device).	<code>README.md:82</code> , <code>.env.example:39-43</code> , <code>src/mmgraphrag/utils/config_utils.py:160-162</code> , <code>MMGraphRAG.py:509</code> [V-read]
CUDA-only	<code>cupy</code> is imported at module import (<code>MMGraphRAG.py:12-13</code>), PPR has no CPU path (<code>:1545-1549</code>), and spaCy calls <code>require_gpu</code> (<code>ner_spacy.py:19</code>).	[V-read]
Missing dependency	The E-VQA builder imports <code>ijson</code> , which is not in <code>requirements.txt</code> .	<code>examples/data/build_evqa_100k.py:9</code> [V-read]

Item	Finding	Evidence
Showcase	examples/showcase was not produced from the bundled input: its relations mention <i>Kaokoland</i> and <i>Namibia</i> , which don't occur in input.json.	examples/showcase/impala/text_structure.json:13 [V-read, V-run grep]
Stubbed end-to-end run (my script)	Works. The repo's own indexing, graph construction, seed building, fusion and ranking ran on the demo, with spaCy, SAM3, EVA-CLIP and CuPy replaced by stubs and PPR re-implemented in numpy with the same equations.	audit/scripts/mock_mg2rag_e2e.py, audit/logs/mock_mg2rag/report.json [V-run]

Real run needs [V-run: HF metadata API; VRAM figures I]:

- EVA-CLIP-8B: four fp32 .bin shards, **32.9 GB** download, about 16 GB VRAM in fp16.
- SAM3: sam3.pt, 3.45 GB, **gated, manual approval**.
- en_core_web_trf: 0.46 GB.
- CLIP ViT-L/14 *image-processor config* only (a few KB).

EVA-CLIP stays resident in the main process while the SAM3 and spaCy subprocesses run on the same device. So a single-GPU run needs a **full A100 (40 GB)**, not the course's free half-A100 MIG slice [I]. The local RTX 5070 (12 GB, Blackwell) cannot hold EVA-CLIP-8B and is not covered by the cu126 wheels [I].

3.3 Download and cost plan (waiting for your go-ahead)

Everything heavy should go to Snellius scratch, not the laptop.

#	What	Size / cost	Needed for
1	BAAI/bge-large-en-v1.5 (safetensors only)	≈1.3 GB	Real MemGraphRAG smoke test
2	LLM calls for that smoke test: 50 HotpotQA passages with gpt-4o-mini, or an open LLM via vLLM	≈1–2k calls, ≈2M tokens, < US\$1 (price per token [I]). Needs <i>your</i> API key. The vLLM route needs e.g. Qwen2.5-7B-Instruct (≈15 GB) and GPU credits.	MemGraphRAG
3	spacy/en_core_web_trf	0.46 GB	MG ² -RAG
4	facebook/sam3 sam3.pt	3.45 GB. Request access now ; approval is manual.	MG ² -RAG / Option A
5	Visual encoder for development: google/siglip2-so400m-patch14-384 (Apache-2.0)	4.5 GB	Option A development
6	BAAI/EVA-CLIP-8B (pytorch .bin shards)	32.9 GB	Faithful MG ² -RAG reproduction / final run
7	MLLM for QA and captions: Qwen/Qwen2.5-VL-7B-Instruct (Apache-2.0), or the 3B variant	16.6 GB (3B: 7.5 GB)	Generation, caption baseline
8	M2KR InfoSeek/E-VQA test questions and passages (MIT)	≈0.1 GB	Multimodal evaluation text side
9	Query and KB images for a ≤1k-query subset, fetched per image. Avoid OVEN (294 GB) and AToMiC (174 GB).	≈0.5–2 GB [I]	Multimodal evaluation image side
10	Full MemGraphRAG index of one bundled multi-hop corpus with gpt-4o-mini	≈66k–149k LLM calls, ≈US\$13–37 per corpus [I; audit/scripts/estimate_indexing_cost.py]	Text regression (subsample to stay ≤US\$10)

Sizes for items 1 and 3–8 come from Hugging Face metadata [V-run]; items 2, 9 and 10 are estimates [I]. Compute estimate: MG²-RAG reports 2.9 h to index 5k documents on 2× RTX 6000 Ada (MG² paper Table 1). On `gpu_a100` at 128 credits/h, expect roughly 400–800 credits for a 5k-document knowledge base [I]. Index 1–2k documents instead to stay well below that.

4. Data structures and interfaces that matter for the merge

	MemGraphRAG	MG ² -RAG
Node IDs	<code>chunk-<md5(text)>, entity-<md5(norm.string)>, type-<md5(<TYPE>>); fact-<md5></code> exists only in the fact store	<code>chunk-..., entity-..., image-<md5(pixels)>; sentence-... and subimg-...</code> exist only in stores
Hashing / normalisation	<code>compute_mdhash_id = MD5 + prefix;</code> <code>text_processing = lower-case and keep only alphanumerics and spaces</code> (<code>utils/misc_utils.py:177, 55</code>)	Identical functions (<code>utils/misc_utils.py:144, 70</code>) plus <code>compute_image_hash (:158)</code>
Graph object	<code>igraph.Graph</code> , undirected; vertex attributes <code>name, layer ∈ {passage, entity, type}, content</code> ; edge attributes <code>weight, type ∈ {entity_relation, passage_entity, entity_to_type, type_relation, entity_similarity}</code> . Saved as GraphML plus <code>graph_from_memory/memory_graph.json</code> (<code>MemGraphRAG.py:925-963</code>)	<code>igraph.Graph</code> , undirected; vertex attributes <code>name, content</code> ; edge attribute <code>weight</code> only (types inferred from ID prefixes). Saved as pickle plus <code>aux_maps.pkl</code> (<code>MMGraphRAG.py:757-774</code>)
Memory	<code>ThreeLayerMemory</code> (<code>schema → fact → passage</code> index lists, JSON) (<code>src/Memory.py:36-65</code>)	none
Embedding stores	<code>EmbeddingStore</code> (<code>parquet: hash_id, content, embedding</code>) for chunk, entity and fact; text encoder (BGE 1024-d / NV-Embed-v2 4096-d)	Same class, copied, plus <code>MultimodalEmbeddingStore</code> (<code>embedding_store.py:203</code>) for chunk, entity, sentence, image and sub-image; EVA-CLIP-8B
Retrieval entry	<code>retrieve() → get_fact_scores → rerank_facts (threshold or DSPy) → graph_search_with_fact_entities (seed vector) → run_ppr (:1038-1160, 2089-2339)</code>	<code>retrieve(question_queries, image_queries) → 8× _batch_torch_search → seed building → run_batch_ppr (:776-1060)</code>
Ranking output	PPR scores of passage nodes → <code>QuerySolution(docs, doc_scores)</code>	PPR scores of chunk nodes → <code>QuerySolution(question, image, docs, doc_scores)</code>
Generation	Text LLM, HippoRAG “Wikipedia Title:” prompt (<code>:1240-1324</code>)	Text LLM or MLLM (query image injected as base64 <code>image_url</code> , <code>prompts/prompt_template_manager.py:110-160</code>)

Consequences for the merge [V-read, I]:

- Entity IDs coincide whenever both sides hash the same normalised string, so MG²-RAG’s image–entity grounding edges can attach directly to MemGraphRAG entity nodes.
- The two systems use **different embedding spaces**. Scores are not comparable across spaces, so visual evidence must enter as a *separate seed channel* with its own weight (MG²-RAG’s λ_v), or everything must move to one multimodal encoder.
- MemGraphRAG’s `prepare_retrieval_objects` asserts that entity + passage + type nodes = all vertices (`:1905`). Any new layer must be added there.
- MemGraphRAG’s retrieval falls back to plain dense passage retrieval when no text fact passes the threshold (`:1100-1105`). Image-only questions such as “what is this animal?” would therefore never reach the graph unless the dispatch is changed.

5. Integration options

Option	Idea	Files / functions changed	Reused from MG ² -RAG	Effort	Risk
A. Visual layer on the memory graph <i>(recommended)</i>	Keep MemGraphRAG's text memory pipeline unchanged. After conflict resolution, add image nodes (per passage) and region evidence (SAM3 crops) linked to entity nodes. Choose prompts from surviving fact entities whose schema type is visual (ontology-guided grounding). At query time, add visual seeds (text or image query → image and region similarities) to the existing fact and passage seeds; run the same igraph PPR.	<code>code/index.py</code> (image manifest; chunk per document to keep chunk↔image links); <code>MemGraphRAG.index_with_memory</code> (new "visual grounding" stage); <code>build_memory_graph / install_memory_graph</code> (layers image, optionally region; edge types <code>passage_image, image_entity</code>); <code>prepare_retrieval_objects</code> (:1905 assertion, image embeddings); <code>retrieve</code> dispatch (:1100-1105) and <code>graph_search_with_fact_entities</code> (visual channel, weights); <code>qa</code> (MLLM prompt); <code>retrieval_dataset_test.py</code> (image queries, doc-ID gold, metrics)	<code>grounding_model/sam3Model.py</code> + vendored <code>sam3/</code> + <code>assets/bpe_simple_vocab_16e6.txt.gz</code> ; <code>embedding_model/eva_clip.py</code> (or a SigLIP wrapper of the same interface); <code>MultimodalEmbeddingStore</code> , <code>compute_image_hash</code> ; seed logic <code>MMGraphRAG.py</code> :932-1001; image-injecting prompt manager and <code>mllm_infoseek_qa</code> template	8–12 person-days code, plus about 5 days experiments	Medium: SAM3 environment, prompt selection, tuning λ_v
B. Multimodal memory	Also make memory multimodal: visual facts (e.g. <code>(entity, depicted_in, image)</code>) with visual schema types, and an MLLM "visual verifier" in conflict detection and resolution (the paper's own future-work item).	Everything in A, plus <code>Memory.py</code> (new fields, serialisation), <code>extract_memory_schema</code> , <code>detect_memory_conflicts</code> , <code>resolve_memory_conflicts</code> and the prompts in <code>prompts/prompt.py</code>	As A, plus the MLLM client <code>mllm/mllm_client.py</code>	3–4 person-weeks	High: MLLM cost and latency, no dataset with visual conflicts to show gains
C. MG ² -RAG as base, MemGraphRAG memory as its text backbone	Replace spaCy extraction in MG ² -RAG with MemGraphRAG's resolved memory (entities, facts, types); keep MG ² -RAG's multi-granularity retrieval and CuPy PPR.	MG ² <code>index()</code> (:240-310), <code>update_graph_stats</code> (:359-431), retrieval seeds (add types); MemGraphRAG used as a library up to <code>resolve_memory_conflicts</code>	Most of MG ² -RAG	6–10 person-days	Medium; but the story becomes "improving MG ² -RAG", not "extending MemGraphRAG"

Option	Idea	Files / functions changed	Reused from MG ² -RAG	Effort	Risk
D. Caption baseline (required)	An MLLM captions every image; captions are appended to their passage (or added as passages); MemGraphRAG runs unchanged.	New preprocessing script only	Prompt manager / MLLM client	1–2 days	Low
E. Single multimodal encoder	Replace MemGraphRAG’s text embedder with a CLIP-type encoder so passages, facts and images share one space; add image nodes.	<code>embedding_model/*</code> , <code>EmbeddingStore</code> callers, plus the graph changes of A	EVA-CLIP wrapper	3–5 days on top of A	Medium-high: CLIP text towers are short-context (77 tokens for EVA-CLIP) → likely text-QA regression [I]. Good as an ablation , not as the main system.

Why A. It touches only the end of MemGraphRAG’s pipeline, so memory, conflict resolution and text QA stay intact and comparable. It reuses MG²-RAG’s most expensive-to-rebuild parts (SAM3 wrapper, CLIP store, seed logic) almost verbatim.

Ontology-guided grounding is also a real improvement over MG²-RAG’s NER-label filter:

- MemGraphRAG’s entities come from LLM triples, so common nouns like “impala” or “antelope” become nodes [V-read: `MemGraphRAG.py:832–845`].
- Its schema layer supplies the entity *types* needed to decide what is visually groundable [I].

Optionally add the paper’s missing Eq. 7 and Eq. 8 seeds (about 1 day; `graph_search_with_fact_entities`) so the text system matches its paper.

6. What evaluation needs, and what exists

Need	MemGraphRAG	MG ² -RAG	To write
Text multi-hop QA data	HotpotQA, 2Wiki, MuSiQue (1k questions each), G-Medical [V-run]	–	A subsampling script, plus a mapping from 256-token chunks back to gold passages (or chunk per passage)
Multimodal KB-VQA data	–	E-VQA builder that needs a 174 GB AToMiC scan [V-read, V-run size]	Subset builder: M2KR InfoSeek/E-VQA test text (≈0.1 GB, MIT) plus per-URL download of the needed query and KB images; 500 queries, 1–5k documents with guaranteed gold (as in MG ² App. A.2). Consider E-VQA two-hop questions to give memory and multi-hop room to matter [I].
Loaders	Grouped/flat question JSON (<code>retrieval_dataset_test.py:28–68</code>)	Demo JSON only	Unified loader: documents + images + queries (+ query image) + gold doc IDs
Retrieval metrics	String-matching Recall@k (<code>evaluation/retrieval_eval.py</code>)	–	Doc-ID Recall@{1,5,10} and MRR

Need	MemGraphRAG	MG ² -RAG	To write
QA metrics	EM, F1 (<code>evaluation/qa_eval.py</code>)	–	Containment accuracy, LLM-judge accuracy (paper metrics), InfoSeek VQA/relaxed accuracy. BEM needs a TF-Hub model, so substitute an LLM judge and report EM or containment [I].
Baselines	HippoRAG-style legacy <code>index()</code> (“w/o memory”) [V-run]	Zero-shot MLLM (<code>test_basemodel=True</code>), dense text or image retrieval, visual pivot, full MG ² -RAG [V-read]	Caption baseline (Option D); ablations of Option A: w/o grounding, NER-label vs ontology-guided prompts, w/o visual seeds, λ_v sweep, Option E encoder swap
Efficiency	Token counts per stage (<code>MemGraphRAG.py:622-631, 786-797</code>) and QA timing	PPR and retrieval timers	Indexing wall-clock and GPU-hours per stage

7. Risks and blockers

Risk	Severity	Evidence	Mitigation
Image data volume. OVEN 294 GB (InfoSeek), AToMiC 174 GB (E-VQA builder), both gated	High	HF metadata API [V-run]	Build a small subset and fetch only the needed images; check Snellius <code>/scratch-nvme/ml-datasets</code> and <code>/projects/2/managed_datasets</code> first (Snellius guide)
SAM3 access. Gated with manual approval; SAM License (“other”); the vendored <code>sam3/</code> copy ships without Meta’s LICENSE file	High (schedule)	HF API <code>gated=manual</code> [V-run]; <code>find MG2-RAG/sam3 -iname “*licen*”</code> empty [V-run]	Request access today. Fallback: GroundingDINO or OWLv2 boxes, or skip regions and ground at image level [I]
Dependency conflicts. torch 2.5.1 vs 2.7.0; transformers 4.45.2 vs 4.49.0; tenacity 8.5 vs ≥ 9 ; vllm Linux-only; <code>gritlm</code> hard import; CuPy and spaCy GPU; Windows igraph can’t read GraphML; Smart App Control blocked a pandas 3 DLL on this laptop	Medium	<code>requirements.txt</code> of both [V-read]; stub runs [V-run]	One Linux venv on Snellius with MG ² -RAG’s pins, no vllm, gritlm installed or patched. MemGraphRAG ran fine on these pins in the stub test [V-run]
GPU memory and budget. EVA-CLIP-8B about 16 GB fp16, plus SAM3, on one device	Medium	Model sizes [V-run]; VRAM [I]	Develop on the free MIG slice with a SigLIP-class encoder; use <code>gpu_a100</code> (128 credits/h) only for final runs
LLM call volume. Schema extraction is one call per fact: 66k–149k calls and US\$13–37 per bundled corpus with gpt-4o-mini	Medium	Call structure [V-run]; counts and price [I]	Subsample corpora (e.g. 200 questions with their passages); cache (MemGraphRAG caches LLM calls in SQLite); or self-host Qwen2.5-7B via vLLM
Paper-code gap in MemGraphRAG (no schema retrieval, Eq. 7, Eq. 8 or paper metrics)	Medium (scientific)	§2.1 [V-read, V-run]	State it explicitly; optionally implement Eq. 7 and 8 (≈ 1 day)
Effect may not show. InfoSeek and E-VQA are largely single-hop entity recognition, where memory and conflict machinery may add little	Medium	[I]	Use E-VQA 2-hop items; report retrieval (R@k) and QA; include the caption baseline to isolate the “native visual” effect

Risk	Severity	Evidence	Mitigation
MG ² -RAG text encoder truncates at 77 tokens	Low–medium	eva_clip.py:98 [V-read]	In Option A, keep MemGraphRAG’s text encoder for text; use CLIP only for images and regions
Licensing of outputs	Low	Both repos MIT [V-read]. EVA-CLIP-8B Apache-2.0; NV-Embed-v2 CC-BY-NC-4.0; SAM3 custom; AToMiC CC-BY-SA-4.0; M2KR MIT [V-run: HF metadata]. Wikimedia image licences vary [I]	Fine for coursework; don’t redistribute images or checkpoints; cite licences in the paper

8. Suggested three-week plan [I]

Week	Goal	Exit criterion
29 Sep – 5 Oct	Snellius venv; real MemGraphRAG smoke test (items 1–2); SAM3 access requested; subset builder and loaders; caption baseline	MemGraphRAG answers 50 HotpotQA questions; 500-query multimodal subset on scratch
6 – 12 Oct	Option A implementation; metrics (doc-ID R@k, containment, LLM judge); MG ² -RAG run on the same subset	End-to-end MemGraphRAG-V on 50 queries
13 – 19 Oct	Full runs and ablations; text regression on 200 HotpotQA/2Wiki questions	Results tables frozen by 19 Oct
20 – 23 Oct	Writing (9-page ACM)	Submission

Appendix A. What I ran

Script / command	What it shows	Output
bash code/run_index.sh, bash code/run_retrieval_test.sh (in audit/work copy, dummy key)	Launch scripts fail on \$PYTHON	this report \$3.1
audit/scripts/mock_memgraphrag_e2e.py	Full MemGraphRAG pipeline with stub LLM and embedder; 3 variants (default, a resolution that modifies a triple, legacy index())	audit/logs/mock_memgraphrag/report.json, *.stdout/stderr.txt
audit/scripts/mock_mg2rag_e2e.py	MG ² -RAG demo with stubbed spaCy, SAM3, EVA-CLIP and CuPy; two NER labelings	audit/logs/mock_mg2rag/report.json
audit/scripts/estimate_indexing_cost.py	LLM calls, tokens and cost per bundled corpus	audit/logs/indexing_cost_estimate.json
HF metadata API queries (JSON only)	Checkpoint and dataset sizes, gating, licences	audit/logs/hf_model_sizes.txt, hf_dataset_sizes.txt

Environment. The run used `audit/.venv`: Python 3.12, torch 2.7.0+cpu, transformers 4.49.0, igraph 0.11.8, pandas 2.2.3 (about 1.9 GB on disk; no model weights).

Stubs replaced only the model calls:

- the LLM,
- the text and CLIP embedders,
- spaCy,
- SAM3,
- CuPy, where PPR was re-implemented in numpy with the same equations.

All scores produced by the stub runs are therefore **not** quality measurements.